

ТЕХНІЧНЕ ПОРІВНЯННЯ JAVASCRIPT І TYPESCRIPT

Питання 3.2.

Технічні зауваження щодо JavaScript

- Кожен браузер має свій власний движок JavaScript.
 - Chrome працює на V8, Firefox використовує SpiderMonkey, а Safari – JavaScriptCore.
 - Ці движки вже не просто інтерпретують JavaScript, а компілюють його в оптимізований машинний код.
 - Поза браузером, середовища виконання, такі як Node.js, також покладаються на V8 для запуску JavaScript на серверах та настільних комп'ютерах.
- Існують інструменти, які намагаються забезпечити строгую типізацію в JavaScript.
 - Flow (від Meta) додає статичну перевірку типів.
 - Бібліотеки середовища виконання, такі як io-ts, дозволяють перевіряти типи під час виконання коду.
 - Але жоден із цих підходів не має такої ж підтримки впровадження чи інструментарію, як TypeScript.
- TypeScript – це надмножина JavaScript, створена Microsoft, вперше запущена у 2012 році.
 - Кожна програма JavaScript є валідною TypeScript (зазвичай).
 - Ви просто отримуєте додаткові інструменти та безпеку.
 - TypeScript не працює безпосередньо в браузері. Його спочатку потрібно скомпілювати (або транспілювати) у JavaScript.

Характеристики TypeScript



- **Самодокументування**

- У TypeScript розробники явно визначають типи даних, що значно підвищує читабельність коду, оскільки розробники розуміють, які типи даних функція очікує на вхід або вихід.

- **Інструменти розробки**

- TypeScript надає функції для покращення взаємодії з розробниками, зокрема: покращений IntelliSense (автозаповнення коду); виведення типів для підвищення безпеки первинного коду; перевірка помилок у реальному часі (на відміну від перевірок під час виконання JavaScript) для спрощення налагодження та ін.

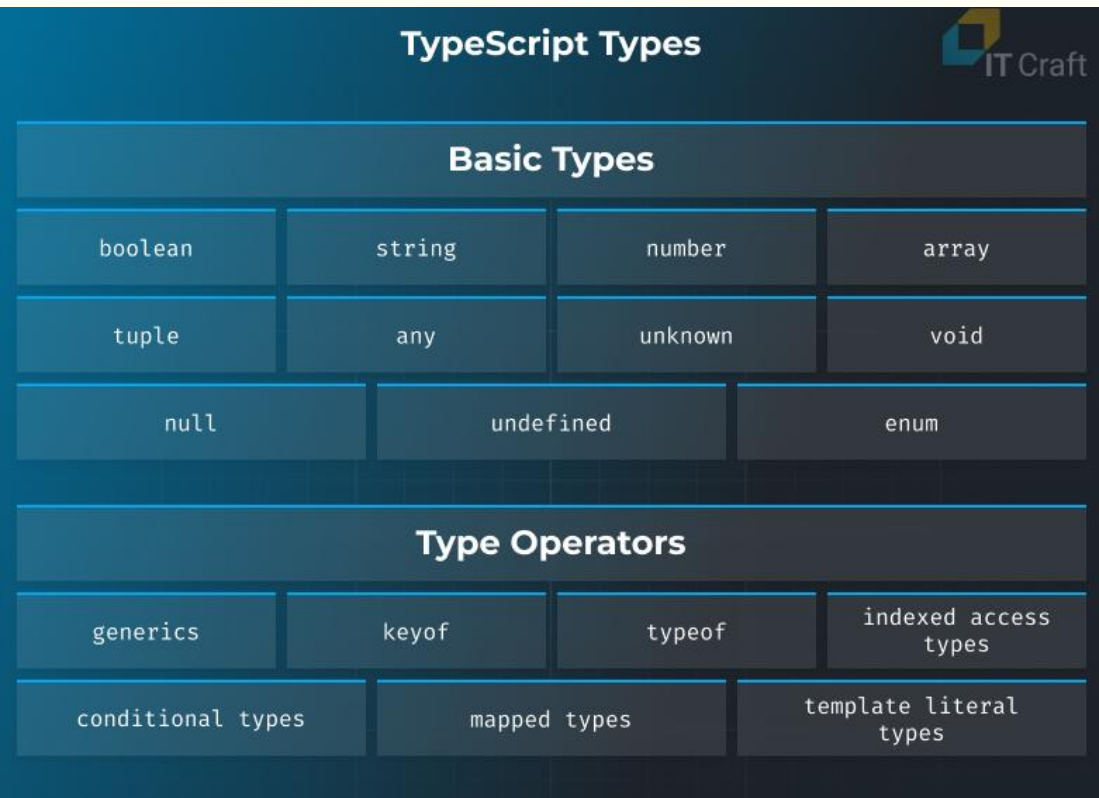
- **Сумісність з JavaScript**

- Кожен код JavaScript автоматично є кодом TypeScript – розробникам потрібно лише перейменувати розширення файлу .js на .ts.
- Хоча просте перейменування файлів не працює у зворотному напрямку, розробники можуть легко перекомпілювати код TypeScript у код JavaScript, а потім запускати його будь-де або додавати до існуючої кодової бази JavaScript.

- **Мультипарадигма**

- Розробники можуть використовувати багато підходів до програмування, включаючи функціональне, об'єктно-орієнтоване та імперативне програмування.

Характеристики TypeScript



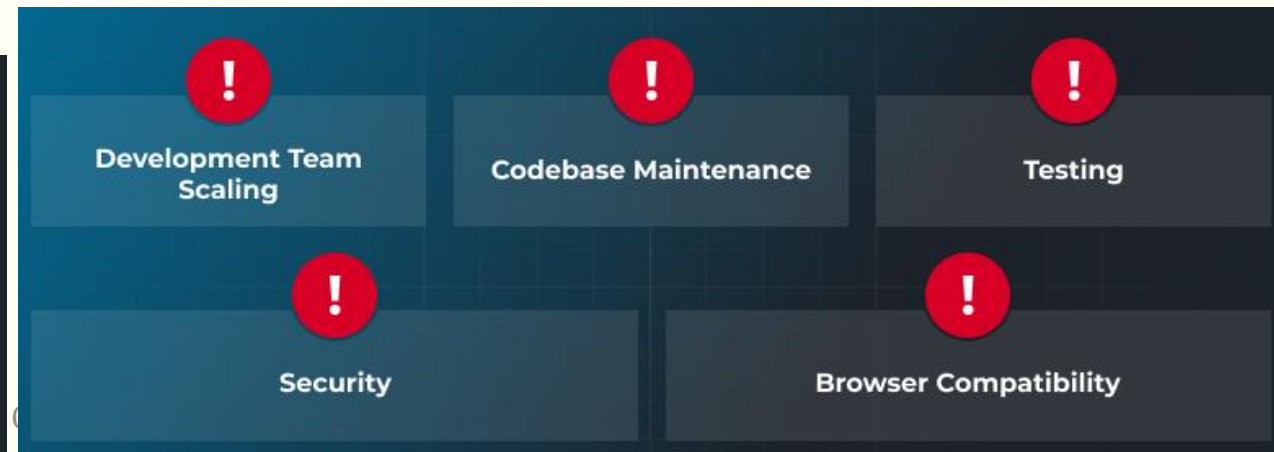
- На відміну від JavaScript, де середовище виконання визначає тип даних динамічно, TypeScript вимагає однозначного оголошення типів.
- Розробники мають два варіанти:
 - TypeScript автоматично визначає тип як значення, коли розробники оголошують змінну.
 - Коли TypeScript не може визначити тип з контексту, розробники повинні призначити тип вручну, якщо їх не влаштовує "будь-який" тип.
- Оператори типів:
 - **generics** використовуються для створення компонентів багаторазового використання, які працюють у різних типах.
 - **keyof** приймає об'єктний тип та створює рядкове або числове літеральне об'єднання його ключів.
 - **typeof** використовується в поєднанні з іншими операторами для створення шаблонів.
 - **типи з індексним доступом** допомагають розробникам шукати певну властивість типу.
 - **умовні (conditional) типи** дозволяють розробникам описувати зв'язки між типами вхідних і вихідних даних.
 - **відображені (mapped) типи** дозволяють розробникам створювати тип на основі іншого типу.
 - **шаблонні (template) літеральні типи** працюють подібно до рядкових літералів у JavaScript та використовуються для зміни властивостей і створення нового типу рядкового літерала шляхом об'єднання вмісту.

JavaScript vs TypeScript

TypeScript



JavaScript



JavaScript vs TypeScript

- Чому варто обрати TypeScript замість JavaScript:
 - Ви зіткнулися з **великим** проектом, що вимагає регулярного додавання нових функцій та ефективного масштабування, для чого вам може знадобитися швидко розширити команду розробників.
 - Ваше веб-ПЗ спирається на складну бізнес-логіку, і раннє виявлення помилок може значно пришвидшити виконання проекту.
 - Ваша команда має досвід у створенні чистого коду TypeScript.
- Чому варто обрати JavaScript замість TypeScript:
 - Використання мови JavaScript без розширення TypeScript може бути корисним для малих та середніх проектів, де терміни та початкові витрати на розробку є критично важливими.
 - Якщо ваш додаток має просту логіку, TypeScript може створювати зайві накладні витрати.
 - Також, якщо команда проекту впевнено та швидко працює з JavaScript, не варто нав'язувати перехід на TypeScript.
- Чому варто обрати гібридний підхід:
 - Ви можете поєднувати TypeScript та JavaScript для підтримки вашої швидкозростаючої кодової бази або роботи над проектом модернізації програмного забезпечення.
 - Ваша команда може поступово переходити з JavaScript на TypeScript або використовувати TypeScript лише там, де це доречно.

Синтаксичне порівняння мов

TypeScript

```
function multiply(a, b) {  
  
    return a * b;  
  
}  
  
console.log(multiply(5, 3));      // 15  
  
console.log(multiply("5", "3")); // "53" – oops, string concat
```

- JavaScript динамічно типізований. Мова сама визначає тип під час виконання коду.
 - Це робить його швидким для написання, але також ризикованим, оскільки проста помилка може порушити роботу вашої програми під час виконання.

JavaScript

```
function multiply(a: number, b: number): number {  
  
    return a * b;  
  
}  
  
console.log(multiply(5, 3));      // 15  
  
console.log(multiply("5", "3")); // Error: Argument of type 'string' is not a  
parameter of type 'number'
```

- Ви можете оголошувати типи змінних, параметрів та значень, що повертаються.
 - Компілятор перевіряє їх перед виконанням коду.

Синтаксичне порівняння мов

TypeScript

```
interface Product {  
  
    id: number;  
  
    name: string;  
  
    price: number;  
  
    description?: string; // optional property  
}  
  
function formatProduct(p: Product): string {  
  
    return `${p.name} - ${p.price}`;  
}
```

- TypeScript також дозволяє визначати інтерфейси та типи для опису форми даних.
 - Це кардинально змінює правила гри, коли ви працюєте зі складними об'єктами.
 - Інтерфейс дає вам чіткий контракт: будь-який об'єкт Product повинен мати ідентифікатор, назву та ціну.
 - Якщо ви забудете щось із цього, компілятор одразу повідомить про це.
 - У великих кодових базах статична типізація зменшує кількість помилок, покращує читабельність та спрощує адаптацію нових розробників.

Синтаксичне порівняння мов. Обробка помилок

TypeScript

```
interface User {  
    name: string;  
}  
  
function getUserName(user: User): string {  
    return user.name.toUpperCase();  
}  
  
console.log(getUserName(undefined));  
  
// Error: Argument of type 'undefined' is not assignable to parameter of
```

- TypeScript виявляє проблеми під час компіляції.
 - Це робить налагодження швидшим, а рефакторинг безпечнішим.
 - Ви витрачаєте менше часу на пошук помилок під час виконання та більше часу на впровадження функцій.

JavaScript

```
function getUserName(user) {  
    return user.name.toUpperCase();  
}  
  
console.log(getUserName(undefined));  
  
// Runtime error: Cannot read properties of undefined
```

- JavaScript повідомляє про помилку лише тоді, коли код фактично виконується.
 - Це нормально для невеликих скриптів, але у великих проєктах це може коштувати вам часу та грошей.
 - Баг не проявиться, доки хтось не натрапить на нього у продакшені або, якщо пощастить, під час тестування.

Інструментальне порівняння мов

```
// Install type definitions separately

// npm install express

// npm install --save-dev @types/express

import express, { Request, Response } from "express";

const app = express();

app.get("/", (req: Request, res: Response) => {

    res.send("Hello, TypeScript + Express!");

});
```

- JavaScript має найбільшу екосистему в програмуванні:
 - Менеджер пакетів npm містить понад 2,3 мільйона пакетів – найбільший реєстр ПЗ у світі.
 - Популярні фреймворки, такі як React, Angular та Vue, написані на JavaScript або побудовані на його основі.
 - Будь-яку бібліотеку JavaScript можна використовувати в TypeScript.
 - TypeScript потребує визначення типів, щоб зрозуміти, що відбувається всередині цих бібліотек.
 - Ці визначення повідомляють компілятору, які типи очікує та повертає бібліотека.
 - Для Express.js без @types/express ви втратите автозаповнення, перевірку на помилки та безпеку типів.
 - Але з ним ваше IDE точно знає, що таке req або res.
 - Більшість основних бібліотек зараз містять вбудовані визначення TypeScript або підтримують їх через DefinitelyTyped.
 - Екосистема, фактично, та ж: різниця полягає в тому, що TypeScript надає вам потужніші інструменти.

Інструментальне порівняння мов

```
interface Car {  
    make: string;  
    model: string;  
    year: number;  
}  
  
const myCar: Car = {  
    make: "Tesla",  
    model: "Model 3",  
    year: 2022,  
};  
  
console.log(myCar);
```

- Сучасні редактори, такі як VS Code або WebStorm, надають автозаповнення та основні підказки щодо помилок, проте без статичних типів ці підказки обмежені.
 - Редактор не завжди може здогадатися, що має повертати ваш код.
- Оскільки компілятор TypeScript знає типи, IDE можуть надати вам розширену підтримку:
 - Автозаповнення: пропонує властивості та методи під час введення тексту.
 - Рефакторинг: безпечне перейменування змінних, методів або файлів у вашому проєкті.
 - Навігація: перехід безпосередньо до визначень та посилань.
 - Перевірка помилок у режимі реального часу: перевіряє невідповідності типів під час написання коду, перш ніж щось виконати.
- У TypeScript ваша IDE автоматично доповнить: `make`, `model`, `year`.
 - У звичайному JavaScript ваша IDE може спробувати вгадати, але вона не може гарантувати правильність.

ООП в мовах JavaScript і TypeScript. Класи

- Класи в JavaScript насправді є «спеціальними функціями», і так само, як можна визначати функціональні вирази та оголошення функцій, клас можна визначити двома способами: виразом класу або оголошенням класу.
 - Оскільки TypeScript є підмножиною JavaScript, він реалізує класи так само, як і JS.
 - Однак, через строго типізовану природу TS, поверх JS додано кілька властивостей, які покращують читабельність, зручність обслуговування та зменшують кількість помилок.

- Класи в JavaScript

```
class Rectangle {  
  constructor(height, width) {  
    this.height = height;  
    this.width = width;  
  }  
}
```

- Немає великої різниці в загальному синтаксисі класів між JS та TS.
 - Класи TypeScript можна вважати більш читабельними, зручними в обслуговуванні та такими, що містять менше помилок.

Класи в TypeScript

```
class Animal {  
  name: string;  
  
  constructor(name: string) {  
    this.name = name;  
  }  
  
  speak(): void {  
    console.log(`${this.name} makes a noise.`);  
  }  
}
```

ООП в мовах JavaScript і TypeScript. Інтерфейси

- Мова JavaScript не підтримує інтерфейси, оскільки її механізми успадкування базуються на об'єктах, а не на класах.
 - JavaScript використовує качину типізацію.
- TypeScript підтримує інтерфейси, ключові слова `interface` і `type`.
 - Вони дозволяють TS застосовувати структуру до об'єктів, гарантуючи, що об'єкти мають очікувані властивості.
 - Це спрощує виявлення помилок під час компіляції та написання більш зручного для підтримки коду.
 - Псевдоніми типів – ще один механізм забезпечення структури об'єктів.

```
// JavaScript
function printLabel(labelObj) {
  console.log(labelObj.label);
}
```

```
// TypeScript
interface LabelObj {
  label: string;
}

function printLabel(labelObj: LabelObj): void {
  console.log(labelObj.label);
}

printLabel({ label: "name" })
```

```
interface Animal {
  name: string
}

interface Bear extends Animal {
  honey: boolean
}

interface Animal {
  favFood: string
}
```

```
type Animal = {
  name: string
}

type Bear = Animal & {
  honey: boolean
}

interface Animal {
  favFood: string
}

// Error: Duplicate identifier 'Animal'.
```

ООП в мовах JavaScript і TypeScript. Модифікатори доступу

- TypeScript також пропонує модифікатори доступу, які дозволяють обмежити доступ до певних властивостей і методів класу.
 - Модифікатори доступу, такі як `private` і `protected`, допомагають забезпечити інкапсуляцію та запобігти несанкціонованому доступу до внутрішнього стану об'єкта.
 - JavaScript не має модифікаторів доступу, що ускладнює інкапсуляцію та підтримку чистоти кодової бази.
- Припустимо, ви хочете створити клас `Person` із закритою властивістю під назвою `age`.
 - Ви хочете створити метод під назвою `getAge()`, який повертає значення властивості `age`, але ви не хочете дозволяти прямий доступ до властивості `age` з-за меж класу.

```
// JavaScript

function Person(age) {
    This.age =age;
}

Person.prototype.getAge = function() {
    Return this.age
}
```

```
// TypeScript
class Person {
    private age: number;

    constructor(age: number) {
        this.age = age;
    }

    public get getAge(): number {
        return this.age;
    }
}
```

ООП в мовах JavaScript і TypeScript. Модифікатори доступу

```
//TypeScript

class Employee {
  private name: string;
  readonly ssn : string;
  private static count: number = 0;

  constructor(name:string, ssn:string){
    this.name = name
    this.ssn = ssn
    Employee.count +=1;
  }
  public getName():string {
    return this.name;
  }
  public static getCount():number {
    return Employee.count;
  }
  public getSSN():string{
    return this.ssn;
  }
};

let emp:Employee = new Employee("Mike", "836527496");

//emp.count; // 'count' does not exist on type 'Employee'
//emp.ssn=5; // Cannot assign to 'ssn' because it is a read-only property
```

- Крім популярних `private`, `public` та `protected`, TypeScript має два додаткові модифікатори доступу: `static` та `readonly`.
 - static-властивості:** до них можна отримати доступ лише з класу, а не з об'єкта, створеного з нього, і їх не можна успадкувати.
 - readonly-властивості:** ці властивості не можна змінювати після ініціалізації, їх можна лише читати. Їх необхідно встановити під час оголошення класу або всередині конструктора.

ООП в мовах JavaScript і TypeScript. Модифікатори доступу

```
//JavaScript

class Time {
  #hour = 0
  #minute = 0
  #second = 0
  static #GMT = 0
  constructor(hour, minute, second) {
    this.#hour = hour
    this.#minute = minute
    this.#second = second
  }
  static get displayGMT() {
    return Time.#GMT
  }

  static set setGMT(gmt){
    Time.#GMT=gmt
  }
  get getHour(){
    return this.#hour
  }
  set setHour(hour){
    this.#hour=hour
  }
}
```

- Як згадувалося раніше, підтримка ключового слова class у JS була запроваджена в ES6, а разом з нею й деякі функції класів:
 - ключове слово static
 - модифікатор приватного доступу з використанням # нотації
 - ключові слова get та set
- Незважаючи на наявність цих функцій, їхня поведінка на перший погляд досить заплутана для розуміння, до того ж, їхня читабельність менша, ніж у еквівалента TypeScript.

```
//console.log(Time.getHour()) // Error: Time.getHour is not a function
console.log(Time.getHour) //undefined
console.log(Time.displayGMT) //0
Time.setGMT = 2
myTime = new Time()
console.log(Time.#hour) //Error: reference to undeclared private field or method
console.log(myTime.#hour) //Error: reference to undeclared private field or method
console.log(Time.displayGMT) //2
```


ООП в мовах JavaScript і TypeScript. Модифікатори доступу

```
//TypeScript

class Time {
  private hour: number = 0
  private minute: number = 0
  private second: number = 0
  private static GMT: number = 0
  constructor(hour: number, minute: number, second: number) {
    this.hour = hour
    this.minute = minute
    this.second = second
  }
  static get displayGMT(): number {
    return Time.GMT
  }

  static set setGMT(gmt: number) {
    Time.GMT = gmt
  }
  get getHour(): number {
    return this.hour
  }
  set setHour(hour: number) {
    this.hour = hour
  }
}
```

■ TypeScript-еквівалент

```
//console.log(Time.getHour()) //Property 'getHour' does not exist on type 'typeof Time'
console.log(Time.displayGMT) //0
Time.setGMT = 2
//myTime = new Time() //Expected 3 arguments, but got 0.
const myTime: Time = new Time(1, 2, 3)
//console.log(Time.hour) //Property 'hour' does not exist on type 'typeof Time'
//console.log(myTime.hour) //Property 'hour' is private and only accessible with the 'private' modifier
console.log(Time.displayGMT) //2
```

ООП в мовах JavaScript і TypeScript. Наслідування та поліморфізм

```
// JavaScript
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  console.log(`${this.name} makes a noise.`);
}

function Dog(name) {
  Animal.call(this, name);
}

Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

Dog.prototype.speak = function() {
  console.log(`${this.name} barks.`);
}
```

- У прикладі JavaScript реалізовано наслідування на основі прототипів, яке може бути складним для читання та підтримки.
 - У TypeScript успадкування реалізовано за допомогою ключового слова `extends`, що робить код більш читабельним та простішим у підтримці.
 - З появою ES6-класів ключові слова `extends` також стали підтримуватися в class-реалізаціях:

```
// TypeScript
class Animal {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  speak(): void {
    console.log(`${this.name} makes a noise.`);
  }
}

class Dog extends Animal {
  constructor(name: string) {
    super(name);
  }

  speak(): void {
    console.log(`${this.name} barks.`);
  }
}
```

```
class Cat {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a noise.`);
  }
}

class Lion extends Cat {
  speak() {
    super.speak();
    console.log(`${this.name} roars.`);
  }
}

const myLion = new Lion("Fuzzy");
// Fuzzy makes a noise.

myLion.speak(); // Fuzzy roars.
```

ООП в мовах JavaScript і TypeScript. Абстрактні класи

- TypeScript підтримує абстрактні класи, тобто класи, екземпляри яких не можна створювати безпосередньо.
 - Немає прямої реалізації абстрактного класу в JS, проте існують способи/обхідні шляхи застосування абстракції. У наведеному прикладі JavaScript ми створили функцію-конструктор, яка викидає помилку, якщо викликається конструктор.
 - Коли справа доходить до підкласу Bike, то використовується його прототип та доступ до функції Object.create(), щоб встановити його на прототип базового класу.

```
// JavaScript
function Vehicle()
{
    this.vehicleName="vehicleName";
    throw new Error("You cannot create an instance of Abstract Class");
}
Vehicle.prototype.display=function()
{
    return "Vehicle is: "+this.vehicleName;
}
//Creating a constructor function
function Bike(vehicleName)
{
    this.vehicleName=vehicleName;
}
//Creating object without using the function constructor
Bike.prototype=Object.create(Vehicle.prototype);
```

```
// TypeScript
abstract class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract speak(): void;
}
```

```
class Dog extends Animal {
    constructor(name: string) {
        super(name);
    }

    speak(): void {
        console.log(`${this.name} barks.`);
    }
}

class Cat extends Animal {
    constructor(name: string) {
        super(name);
    }

    speak(): void {
        console.log(`${this.name} meows.`);
    }
}
```

ООП в мовах JavaScript і TypeScript. Дженерики (узагальнені типи)

```
// TypeScript
function identity<T>(arg: T): T {
  return arg;
}

let output1 = identity<string>("Hello");
let output2 = identity<number>(42);
```

```
// JavaScript
function identity(arg) {
  return arg;
}

let output1 = identity("Hello");
let output2 = identity(42);
```

```
//JavaScript

function getArray(arr) {
  return new Array().concat(arr);
}

let myNumArr = getArray([10, 20, 30]);
let myStrArr = getArray(["Hello", "JavaScript"]);
myNumArr.push(40); // Correct
myNumArr.push("Hello OOP"); // Correct
myStrArr.push("Hello TypeScript"); // Correct
myStrArr.push(40); // Correct
console.log(myNumArr); // [10, 20, 30, 40, "Hello OOP"]
console.log(myStrArr); // ["Hello", "JavaScript", "Hello TypeScript", 40]
```

- TypeScript також підтримує узагальнення, що дозволяє створювати код повторного використання, який може працювати з різними типами даних.
 - Узагальнення особливо корисні під час роботи з колекціями, такими як масиви, де вам може знадобитися визначити функцію або клас, який може працювати з будь-яким типом даних.
 - JavaScript не має узагальнень, що ускладнює написання узагальненого коду.
 - Однак, цей приклад можна легко створити на JS з подібними результатами, тому він насправді не показує, чому дженерики важливі та чому їхня відсутність у JavaScript ускладнює написання повторно використовованого строго типізованого коду.
- У наведеному прикладі функція `getArray()` приймає масив.
- Функція `getArray()` створює новий масив, об'єднує з ним елементи та повертає цей новий масив.
- Оскільки JavaScript не визначає застосування типів даних, ми можемо передавати функції елементи будь-якого типу.
- Але нам потрібно додавати числа до `myNumArr` та рядки до `myStrArr`, ми не хочемо додавати числа до `myStrArr` або навпаки.

ООП в мовах JavaScript і TypeScript. Дженерики (узагальнені типи)

```
// TypeScript
function identity<T>(arg: T): T {
  return arg;
}

let output1 = identity<string>("Hello");
let output2 = identity<number>(42);
```

- Щоб вирішити цю проблему, TypeScript запровадив дженерики.
 - В узагальненнях змінна типу приймає лише той тип, який користувач вказує під час оголошення.

```
function getArray<T>(arr: T[]): T[] {
  return new Array<T>().concat(arr);
}

let myNumArr = getArray<number>([10, 20, 30]);
let myStrArr = getArray<string>(["Hello", "JavaScript"]);
myNumArr.push(40); // Correct
myNumArr.push("Hi! OOP"); // Error: Argument of type 'string' is not assignable
myStrArr.push("Hello TypeScript"); // Correct
myStrArr.push(50); // Error: Argument of type 'number' is not assignable to para

console.log(myNumArr);
console.log(myStrArr);
```

ООП в мовах JavaScript і TypeScript. Типи Intersection та Union

```
// TypeScript
interface Dog {
  name: string;
  breed: string;
}

interface Cat {
  name: string;
  color: string;
}

type Pet = Dog & Cat; // Intersection
type DogOrCat = Dog | Cat; // Union

let myPet: Pet = { name: "Fluffy", breed: "Poodle", color: "White" };
// Error: Property 'color' is missing in type 'Pet' but required in type 'Cat'.

let myDogOrCat: DogOrCat = { name: "Mittens", breed: "Tabby" };
```

- TypeScript підтримує типи для перетину та об'єднання, які є способом створення нових типів.
 - Типи Intersection дозволяють розробникам створювати типи, які мають усі властивості та методи двох або більше типів.
 - Типи Union дозволяють розробникам створювати типи, які можуть бути одного з двох або більше типів.
- У наведеному прикладі інтерфейси Dog та Cat визначені з різними властивостями.
 - Тип Pet визначається як перетин інтерфейсів Dog та Cat: він має спільні властивості обох інтерфейсів.
 - Тип DogOrCat визначається як об'єднання інтерфейсів Dog та Cat: він може бути або собакою, або котом.
 - Змінній myPet присвоюється об'єкт, який має всі властивості обох інтерфейсів Dog та Cat, але TypeScript видає помилку компілятора, оскільки відсутня властивість color.
 - Змінній myDogOrCat присвоюється об'єкт, який має властивості інтерфейсу Dog.

ООП в мовах JavaScript і TypeScript. Декоратори

- TypeScript підтримує декоратори, які є способом додавання метаданих до класів, методів та властивостей.
 - Декоратори дозволяють розробникам писати код, який є більш декларативним та гнучким.
 - На основі цієї функції побудовано кілька бібліотек та фреймворків, наприклад, Angular та Nest.js.
 - У наведеному прикладі функція `myDecorator()` визначена з трьома параметрами: `target`(конструктор класу), `propertyKey`(назва декорованого методу), `descriptor`(об'єкт, що описує декорований метод).
 - Метод `myMethod()` класу `MyClass` декорується функцією `myDecorator`.
 - Коли метод `myMethod()` викликається, декоратор виводить повідомлення в консоль.

```
function myDecorator(target: any, propertyKey: string, descriptor: PropertyDescriptor) {  
    console.log(`Decorating ${target.constructor.name}.${propertyKey}`);  
}  
  
class MyClass {  
    @myDecorator  
    myMethod() {  
        console.log("Hello, world!");  
    }  
}
```


ООП в мовах JavaScript і TypeScript. Декоратори

```
import "reflect-metadata";
const requiredMetadataKey = Symbol("required");
function required(target: Object, propertyKey: string | symbol, parameterIndex: number) {
  let existingRequiredParameters: number[] = Reflect.getOwnMetadata(requiredMetadataKey, target, propertyKey) || [];
  existingRequiredParameters.push(parameterIndex);
  Reflect.defineMetadata(requiredMetadataKey, existingRequiredParameters, target, propertyKey);
}
function validate(target: any, propertyName: string, descriptor: TypedPropertyDescriptor<Function>) {
  let method = descriptor.value!;
  descriptor.value = function () {
    let requiredParameters: number[] = Reflect.getOwnMetadata(requiredMetadataKey, target, propertyName);
    console.log("params", requiredParameters);
    if (requiredParameters) {
      for (let parameterIndex of requiredParameters) {
        if (parameterIndex >= arguments.length || arguments[parameterIndex] === undefined) {
          throw new Error("Missing required argument.");
        }
      }
    }
  };
  return method.apply(this, arguments);
};
```

- Розглянемо більш складний приклад, який показує реальний випадок використання декоратора.
 - У цьому прикладі використовується декоратор параметрів та декоратор методів.
 - Декоратор `@required` додає запис метаданих, який позначає параметр як обов'язковий.
 - Потім декоратор `@validate` обгортає існуючий метод функцією `greet()`, яка перевіряє аргументи перед викликом оригінального методу.

ООП в мовах JavaScript і TypeScript. Декоратори

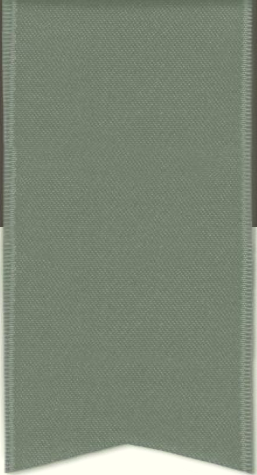
```
class BugReport {
  type = "report";
  title: "string";
  constructor(t: string) {
    this.title = t;
  }

  @validate
  print(@required verbose: boolean) {
    if (verbose) {
      return `type: ${this.type}\ntitle: "${this.title}`;
    } else {
      return this.title;
    }
  }
}

const br = new BugReport("Bug Report Message")

console.log(br.print()) // [ERR]: Missing required argument.
console.log(br.print(true)) // [LOG]: "type: report title: "Bug Report Message\""
```

- Декоратори є експериментальною функцією в TypeScript.
 - Щоб їх використовувати, вам потрібно встановити ціль компілятора на tsconfig.tsES5 або вище та ввімкнути experimentalDecorators.
 - Крім того, у прикладі вам також потрібно буде встановити бібліотеку reflect-metadata та ввімкнути emitDecoratorMetadata у файлі .tsconfig.ts.
 - Актуальна документація.



ДЯКУЮ ЗА УВАГУ!

Наступне питання: Об'єктно-орієнтоване програмування в клієнтських вебзастосунках.